

Reward-Weighted Behavior Cloning for Flow-Matching Vision-Language-Action Policies: A Dual-Suite Null on LIBERO

Gaurav Mitra
Georgia Institute of Technology*

Abstract

Supervised fine-tuning (SFT) of vision-language-action (VLA) models tends to plateau, and reinforcement learning (RL) is the obvious next step. Modern VLAs, though, emit actions through a flow-matching denoiser with no tractable per-action log-probability, so PPO does not apply. We study a lightweight alternative that fits flow matching: filtered, reward-weighted behavior cloning on a policy’s own successful rollouts, regularized toward the SFT policy by a vector-field distillation anchor that stands in for an intractable marginal KL. Using SmolVLA (450M) on LIBERO and a compute-frugal pipeline (a consumer GPU plus preemptible cloud, about \$20–60 total), we evaluate the method in two regimes under a paired *collection-equals-evaluation* protocol: the saturated, in-distribution LIBERO-Goal suite (about 89%) and an unsaturated, demo-seeded `libero_10` (LIBERO-Long, about 44%). In neither regime does the method improve the policy durably. Success rises for one iteration and returns to baseline the next (89.0 $\rightarrow$ 91.0 $\rightarrow$ 89.4 on LIBERO-Goal; 44.0 $\rightarrow$ 47.5 $\rightarrow$ 42.5 on `libero_10`), a within-noise redistribution rather than a learned gain. Two mechanisms explain the pattern: a marginal-task rescue-then-reversal, and a starvation floor for tasks with zero seed successes. The null is informative because it holds even with real headroom and no filter starvation. The contribution is the method, the paired protocol, a map of where lightweight RL can and cannot move a flow-matching VLA, and the dual-suite null itself.

1 Introduction

Generalist robot manipulation increasingly relies on *vision-language-action* (VLA) models: a single policy mapping camera images and a natural-language instruction to motor actions across many tasks [6, 16]. Two facts shape the research loop. First, the strongest VLAs are large (OpenVLA is 7B [6]), which puts them out of reach of modest compute. Second, improving a VLA with reinforcement learning is unsolved for the modern variants whose action head is a flow-matching or diffusion model [4, 8]: such a head produces actions by iteratively denoising noise, so the marginal likelihood of a sampled action chunk is intractable and PPO’s probability ratio [15] cannot be formed.

SmolVLA [16] addresses the first fact. At 450M parameters (about 100M of them a trainable action expert, with a frozen VLM backbone) it is competitive on LIBERO while training on a single consumer-class GPU. That makes the whole SFT-to-RL loop affordable, and turns the second fact into a concrete, runnable question:

*Correspondence: mitragaurav10@gmail.com. Code and all result files: <https://github.com/Gogo2015/VLATune>.

Can a lightweight, flow-matching-compatible RL step (filtered, reward-weighted behavior cloning on the policy’s own successes, anchored to the SFT policy by a vector-field trust-region) improve a SmolVLA VLA beyond supervised fine-tuning on LIBERO?

We pick an objective that never needs an action log-probability: reward-weighted regression [12, 13], adapted to flow matching [5]. This sidesteps the intractable-likelihood problem instead of working around it, as the noise-injection and two-layer-MDP methods do [3, 10, 14, 17]. We then test the method as carefully as the compute budget allows and report what we find.

Finding. Across both a saturated in-distribution suite and an unsaturated but bootstrappable one, the method does not improve the policy durably. Aggregate success rises in the first RL iteration, and the second erases it; the per-task gains reverse on the same tasks that drove them. The up-then-down trajectory is itself the evidence that the bump is noise, not signal.

Contributions.

1. A tractable, flow-matching-native RL recipe (reward-weighted filtered BC with a vector-field distillation anchor as a KL surrogate), built as a custom fine-tune step on top of SmolVLA and validated by a gated smoke ladder (Section 3).
2. A paired *collection-equals-evaluation* protocol that turns each iteration’s rollout collection into that policy’s per-task evaluation at no extra cost, with identical initial states across iterations (Section 3).
3. A dual-suite null: no durable gain on saturated LIBERO-Goal or on demo-seeded `libero_10`. This is stronger than a single-suite null, because it holds with real headroom and no filter starvation (Section 5).
4. Two mechanisms behind the null, rescue-then-reversal of marginal tasks and a starvation floor, and a map of where the method can and cannot move the policy (Section 6).

2 Background and Related Work

SmolVLA and flow-matching action experts. SmolVLA [16] pairs a compact vision-language backbone with a small (about 100M) transformer action expert that produces an action *chunk* via flow matching [8]: starting from Gaussian noise, it integrates a learned velocity field to a clean action. Training regresses the predicted velocity onto the noise-to-action target, a supervised MSE with no normalizing constant and no marginal action likelihood. During fine-tuning the backbone is frozen and only the action expert is trained, which is what makes the model data- and compute-efficient.

LIBERO. LIBERO [9] is a simulated (MuJoCo) manipulation benchmark with sub-suites including Goal (10 tasks that share a scene but vary the instruction, isolating goal understanding) and Long (`libero_10`, 10 long-horizon multi-step tasks). Success is outcome-only and binary: the simulator checks a ground-truth goal predicate at the step limit. The paper reports 92% for SmolVLA on LIBERO-Goal, while independent reproductions in the public LeRobot codebase [1] land near 78–84% [2]. We plan around that gap rather than chase it.

Why not PPO; reward-weighted regression. PPO [15] needs the ratio $\pi_\theta(a \mid s)/\pi_{\text{old}}(a \mid s)$, and so a tractable action likelihood, which a flow-matching policy does not provide cheaply. Recent work recovers a likelihood to run on-policy policy gradient, either by injecting noise for a per-step Gaussian log-prob (ReinFlow [17], π RL [3], Flow-GRPO [10]) or by treating the denoising chain as an inner MDP (DPPO [14]). We take the other route and choose an objective that never needs one: reward-weighted, or advantage-weighted, regression [12, 13], whose flow-matching instantiation is online reward-weighted fine-tuning of flow matching [5]. RL does move flow and VLA policies in other settings, often on generalization [3, 7, 11]; our question is whether the lightweight, likelihood-free version moves *this* policy on LIBERO.

3 Method

3.1 Supervised fine-tuning (the RL launch point)

We fine-tune `lerobot/smolvla_base` on the LIBERO-Goal episodes of the `HuggingFaceVLA/libero` dataset (428 usable episodes, batch 64, with the VLM backbone frozen by default). Five non-obvious fixes were needed, and we document them for reproducibility: (i) disabling the Hub push that otherwise aborts training; (ii) working around a broken episode-to-file index in the dataset, which forces a full download and then an in-row `episode_index` filter; (iii) a camera-key contract (`image/image2` to `camera1/camera2`) via `rename_map`; (iv) the same `rename_map` at evaluation; and (v) pre-creating the LIBERO config to avoid an `EOFError` in headless subprocesses. The run was preempted at 16k of 20k steps with converged loss (about 0.10); we evaluate the 16k checkpoint and pin it as the immutable RL reference π_{SFT} .

3.2 The RL loop: rollout, filter, reward-weighted fine-tune

One outer iteration, starting from the current policy π_k (with $\pi_0 = \pi_{\text{SFT}}$):

1. **Collect.** Run N rollouts per task in LIBERO under the exact eval contract (relative control, `n_action_steps=10`, `rename_map`, `MUJOCO_GL=egl`). Record each step’s observation and executed action chunk, the terminal binary success $r \in \{0, 1\}$, and the step count.
2. **Filter and weight.** Keep (obs, action) pairs from successful trajectories. Weight each by a time-to-success factor $w = \exp(-\text{steps}/\tau_t)$, where the per-task τ_t is the median success length, normalized globally to mean 1.
3. **Fine-tune.** Continue-train π_k on the kept set with the flow-matching loss scaled per-sample by w , plus the anchor below, at a small learning rate (2×10^{-5}) for 1000 steps, giving π_{k+1} .
4. **Gate.** Evaluate π_{k+1} and stop early on regression.

The sparse-binary collapse. With $r \in \{0, 1\}$ and no other signal, the return is binary, so reward-weighting is mathematically identical to *filtering*: keep successes, drop failures, behavior-clone the rest. Under sparse binary reward our “reward-weighted BC” is rejection-sampling fine-tuning, the robotics analogue of best-of- N followed by SFT on the passes. The time-to-success weight w is the one cheap, non-degenerate reward shape that makes the weighting more than a filter, and we do not overstate it. What makes this RL rather than more SFT is the signal source: the environment’s reward on the policy’s own rollouts, not human demonstrations.

The vector-field anchor (a tractable KL surrogate). The exact marginal $\text{KL}(\pi_\theta \| \pi_{\text{SFT}})$ over action chunks is intractable for the same reason PPO is. We anchor the mechanism instead. Keeping a frozen copy of π_{SFT} , we penalize the squared distance between the two velocity fields on identical noised inputs,

$$R_{\text{vf}}(\theta) = \mathbb{E}_{s \sim \text{rollout}, t, \varepsilon} \|v_\theta(x_t, t | s) - v_{\text{SFT}}(x_t, t | s)\|^2, \quad (1)$$

added to the loss with weight β . This is the tractable analogue of the per-denoising-step KL that noise-space methods get for free, and it costs one frozen forward pass. Because the live and frozen policies share normalization and see the same (ε, t) , the penalty is exactly 0 at initialization and grows only as the policy drifts. The total per-iteration objective is

$$\mathcal{L}(\theta) = \underbrace{\mathbb{E}_b[w_b \cdot \ell_{\text{fm}}(x_b; \theta)]}_{\text{reward-weighted flow-matching BC}} + \beta R_{\text{vf}}(\theta), \quad (2)$$

with ℓ_{fm} the per-sample SmolVLA flow-matching MSE. We hold the anchor fixed at π_0 across all iterations and use $\beta = 10$ (a smoke sweep showed $\beta = 50$ over-restrains, and $\beta = 0$ is the pure-filtering baseline). The implementation captures v_θ as the output of `action_out_proj` via a forward hook, with no reimplementing of the forward pass. There is also an AWR reading: the reference pull is partly implicit in fitting weighted samples drawn from a policy that starts as π_{SFT} , and the explicit anchor holds it across iterations as π_k drifts.

The demo-seed remedy. Filtered BC needs some successes to bootstrap. A goal-only policy on `libero_10` scores about 0% zero-shot (Section 5), so the filter starves. The remedy is to first SFT a `libero_10` specialist from `smolvla_base`, rather than from the goal SFT, which avoids a goal-transfer confound and lifts the policy off the floor so the loop has something to work with.

The collection-equals-evaluation protocol. The rollout success predicate equals LeRobot’s `pc_success` (we validate this as a smoke gate), so each iteration’s collection *is* that policy’s per-task evaluation. Seeding rollouts deterministically (from 1000) makes the per-task numbers a paired evaluation: π_0 , π_1 , and π_2 see identical initial states, so per-task deltas are not confounded by initialization. This yields a paired $n=500$ evaluation on LIBERO-Goal (50 per task) and $n=20$ per task on `libero_10` at no extra cost.

4 Experimental Setup

All runs use `lerobot 0.5.1`, `torch 2.10.0+cu128`, `transformers 5.3.0`, `python 3.12.13`. Hardware is matched to the bottleneck: an A100-80GB for the GPU-bound SFT and fine-tune steps, and an L4 for the sim/CPU-bound rollouts and evaluation, where a larger GPU only idles through MuJoCo. Evaluation uses relative control, `n_action_steps=10`, the camera `rename_map`, and one process per task; a multi-task eval crashes at the task transition with an EGL context-teardown error on long-running runtimes. The eval CI at $n=100$ is about ± 7 –10 points once within-task clustering is accounted for, so we report per-task throughout and treat $n=500$ as the tighter, paired measurement. Total project compute fits a \$20–60 budget on preemptible Colab Pro+, with per-2000-step checkpointing to Drive for preemption resilience.

Table 1: Headline results. All success rates are `pc_success` (%). RL trajectories are paired (identical seeds across iterations). The `libero_10` SFT figure (50.0, $n=100$) and its paired RL baseline ($\pi_0 = 44.0$, $n=20$ per task, different seeds) measure the same checkpoint under different protocols.

Stage	Suite	n	Result
Official baseline	LIBERO-Goal	100	85.0
Goal SFT (ours, 16k/20k)	LIBERO-Goal	100	89.0 (+4.0, within noise)
RL (filtered BC, $\beta=10$)	LIBERO-Goal	500 (paired)	89.0 $\rightarrow$ 91.0 $\rightarrow$ 89.4
Zero-shot probe (goal-only)	<code>libero_10</code>	10/task	~ 0 (starvation)
Demo-seed SFT	<code>libero_10</code>	100	50.0
RL (filtered BC, $\beta=10$)	<code>libero_10</code>	20/task (paired)	44.0 $\rightarrow$ 47.5 $\rightarrow$ 42.5

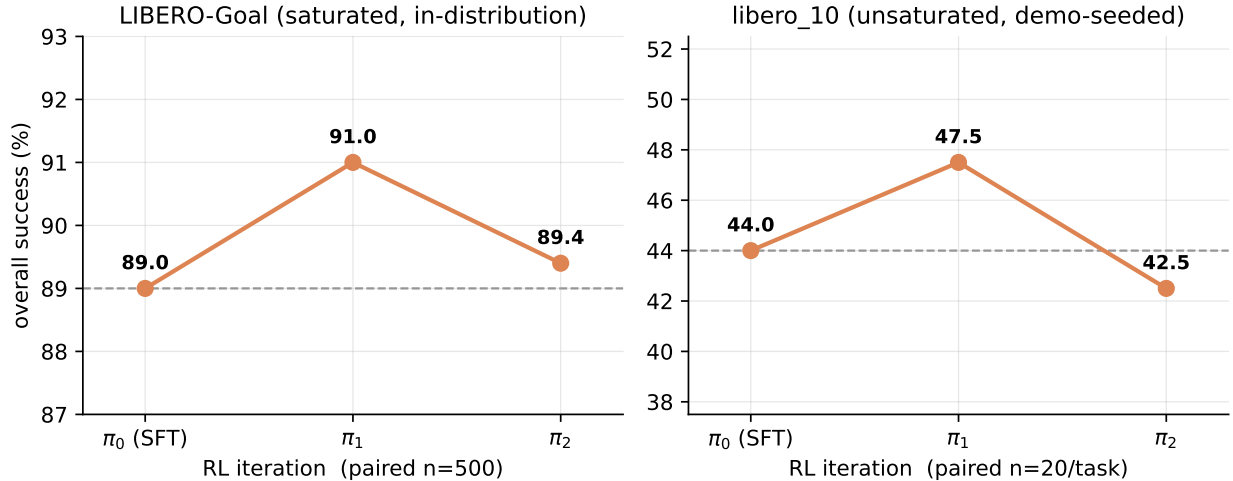


Figure 1: Overall success across RL iterations. In both regimes the trajectory rises one iteration and returns to (or below) the π_0 baseline (dashed) the next, so compounding is falsified. *Left*: saturated LIBERO-Goal (paired $n=500$). *Right*: unsaturated, demo-seeded `libero_10` (paired $n=20$ per task).

5 Results

Table 1 collects the headline numbers; Figures 1–3 show the trajectories, the per-task structure, and the SFT contribution.

5.1 Baseline and SFT (validated pipeline)

The official checkpoint scores 85.0% on LIBERO-Goal ($n=100$), and no task scores 0% (the floor is 60%), which rules out the control-mode and camera-key failure modes that would crater every task uniformly. Our goal-only SFT reaches 89.0% (Figure 3, left). The +4.0 is directionally consistent with goal specialization but sits inside the $n=100$ CI, so it is not a claimed improvement. The SFT deliverables are a validated end-to-end pipeline and a clean checkpoint π_{SFT} to launch RL from.

5.2 RL on saturated LIBERO-Goal: within-noise redistribution

The paired $n=500$ trajectory is 89.0 $\rightarrow$ 91.0 $\rightarrow$ 89.4 (Figure 1, left). Iteration 1 looks positive (+2.0), concentrated on the low-headroom tasks (task 9 78 $\rightarrow$ 90, task 4 88 $\rightarrow$ 100). Iteration 2

does not compound; it regresses -1.6 and erases those gains (task 9 $90 \rightarrow 80$, task 3 $86 \rightarrow 78$). π_2 is statistically indistinguishable from π_0 ($+0.4$). On a near-saturated suite the method redistributes success across tasks within noise rather than producing a durable aggregate gain (Figure 2, top).

5.3 Zero-shot libero_10: the starvation failure mode

Evaluated zero-shot on `libero_10`, the goal-only π_0 scores 0/10 on each probed task. With zero successful rollouts there is nothing for filtered BC to imitate, so the loop cannot bootstrap. This is the sparse-reward starvation failure, and it appears exactly where the headroom is largest. The slowness corroborates it: every episode runs to the long-horizon timeout.

5.4 Demo-seed remedy and RL on libero_10

A `libero_10` specialist SFT lifts success from about 0% to 50.0% ($n=100$; Figure 3, right), which removes the starvation. Running the same RL loop on this genuine headroom reproduces the LIBERO-Goal pattern: paired $44.0 \rightarrow 47.5 \rightarrow 42.5$ (Figure 1, right). Iteration 1’s $+3.5$ is a marginal-task rescue, concentrated on tasks π_0 seeded with a few successes (task 3 $60 \rightarrow 75$, task 7 $25 \rightarrow 40$, task 4 $10 \rightarrow 20$); iteration 2 reverses exactly those tasks (task 3 to 55, task 7 to 25, task 4 to 10), landing -1.5 below π_0 . Establishing the same null on an unsaturated, bootstrappable suite makes it stronger and more general than LIBERO-Goal alone.

6 Discussion

Two mechanisms. The per-task data show why the aggregate does not move durably. First, rescue-then-reversal: lightweight RL transiently lifts weak tasks that the SFT policy seeded with at least a few successes, because the filter then has something to amplify, but the next iteration collects from an already-drifted policy under fixed seeds and reverses them. The up-then-down per-task signature is regression to the mean, or stochastic redistribution, not a learned gain. Second, the starvation floor: a task with zero seed successes (`libero_10` task 8) cannot be bootstrapped by a filter at all and stays at 0 across iterations. The lone iteration-2 “10” is 2/20 seed luck rather than a bootstrap, since π_1 had no task-8 successes to imitate.

Why the null is informative. A negative result is most useful when an obvious confound does not explain it away. This one survives two natural objections. It is not just a ceiling effect, because it also holds on unsaturated `libero_10` with real headroom, and it is not just starvation, because the demo-seed removes starvation and the gain still does not compound. What lightweight RL produces is a redistribution of success across tasks within measurement noise, in both regimes tested.

Limitations. The main ones. (i) Single seed per policy: the per-iteration swings are about 1 SD, so it is the up-then-down trajectory, not any single delta, that reads as noise. (ii) The `libero_10` measurement is $n=20$ per task (binomial CI about ± 20), so the reliable figure is the aggregate. (iii) A single anchor strength $\beta=10$ rather than a full sweep. (iv) RL starts from a 16k (under-trained) SFT base, but the SFT-to-RL delta is measured against that same fixed base on both sides, so it does not contaminate the RL delta. (v) Sim-only, outcome-only binary reward; under it, “reward-weighted” BC equals filtered BC plus a mild time-to-success weight. (vi) Few outer iterations. A claim-grade result would need $n=500$ with at least three seeds varying both training and evaluation seeds, tens of hours on `libero_10`’s slow renderer, which we scope out and flag rather than approximate.

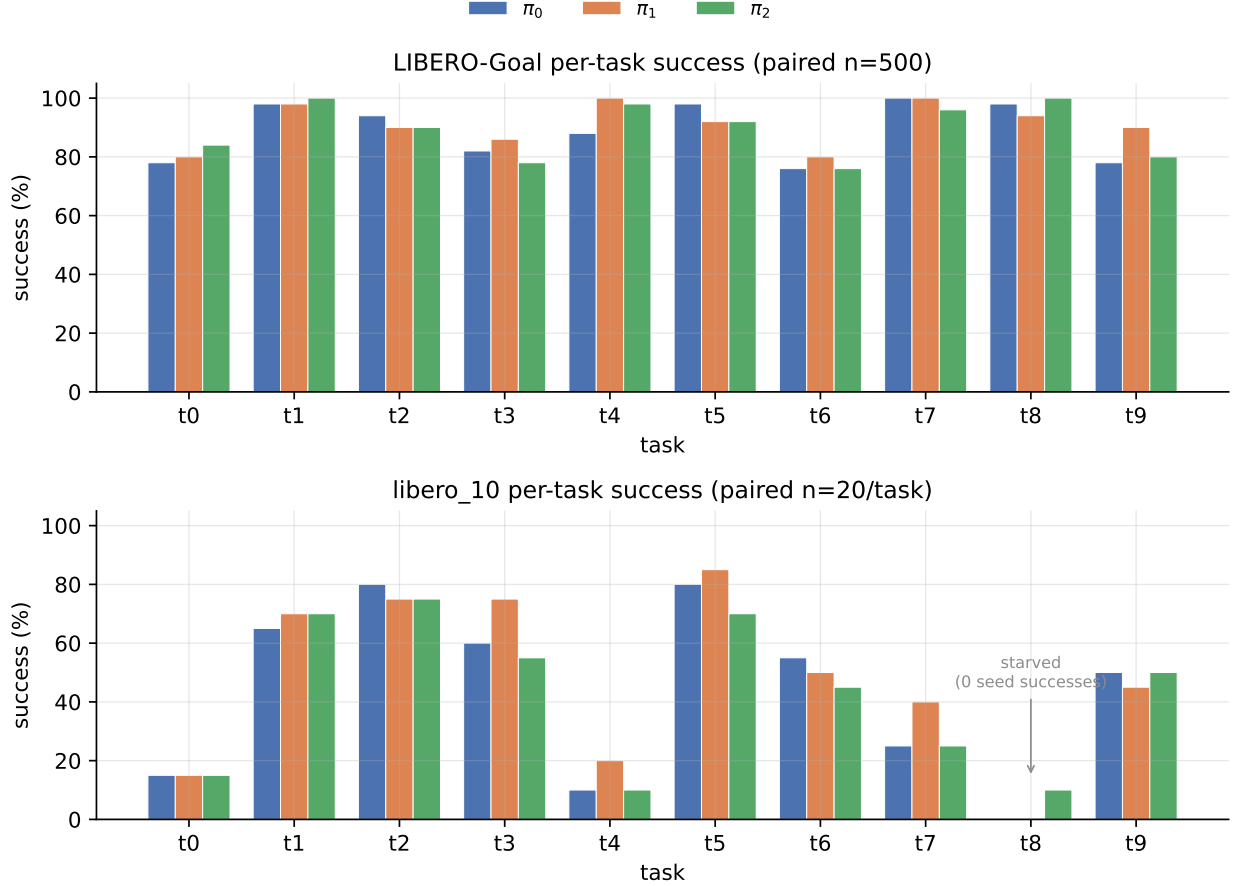


Figure 2: Per-task success for $\pi_0/\pi_1/\pi_2$. *Top*: LIBERO-Goal ($n=500$); gains and losses cancel across tasks. *Bottom*: libero_10 ($n=20$ per task); the marginal tasks rescued in iteration 1 (t3, t4, t7) reverse in iteration 2, while the fully-starved task 8 (0 seed successes) stays on the floor.

7 Conclusion and Future Work

We asked whether a tractable, flow-matching-compatible RL step (reward-weighted filtered BC with a vector-field trust-region anchor) improves a SmolVLA VLA beyond SFT on LIBERO, and tested it as carefully as a \$20–60 budget allows. Across a saturated in-distribution suite and an unsaturated demo-seeded one, the answer is no: success redistributes within noise rather than improving durably, through a rescue-then-reversal mechanism and a starvation floor. The deliverables are the method, the paired collection-equals-evaluation protocol, and a map of where this class of method can and cannot move a flow-matching policy. Natural next steps are a claim-grade evaluation ($n=500$, at least three seeds) to put a confidence interval on the flat delta; an anchor-strength (β) sweep; flow-matching-native on-policy methods that recover a tractable likelihood [3, 10, 14, 17], which target generalization rather than in-distribution success; and real-robot validation. The null, with its mechanisms, is the result.

Reproducibility. All six result files (`results/*.json`) are released with the exact per-task numbers, together with the figure-generation script, the three RL scripts (rollout, fine-tune, eval), and the SFT runbook with the five fixes. The deterministic list of trained episode indices accompanies

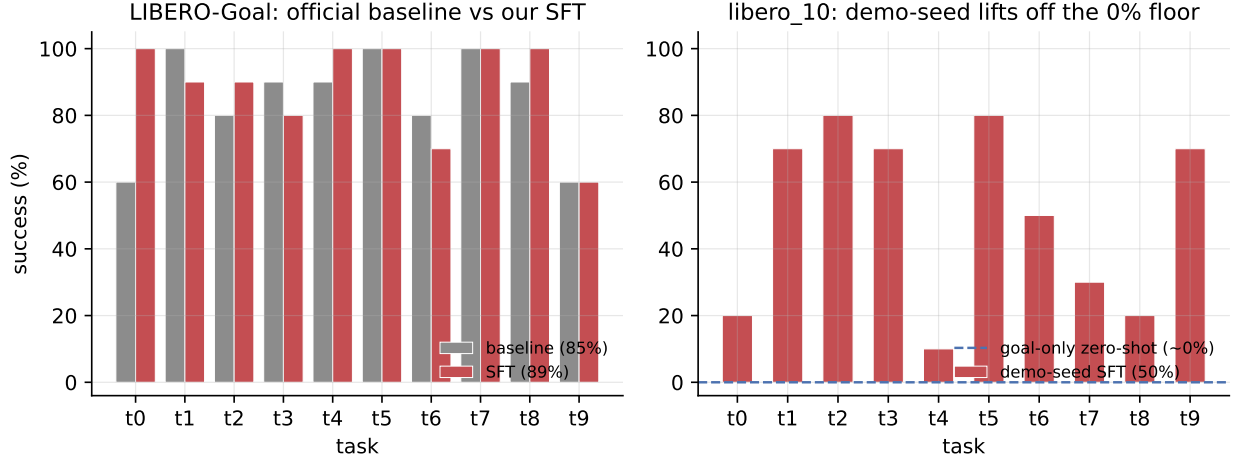


Figure 3: The SFT contribution. *Left*: goal-only SFT vs. the official baseline on LIBERO-Goal (+4.0, within noise). *Right*: the demo-seed SFT lifts `libero_10` off the $\sim 0\%$ goal-only zero-shot floor to 50.0%, which enables, but does not guarantee, an RL effect.

the checkpoints.

References

- [1] Remi Cadene, Simon Alibert, Adil Soare, Quentin Gallouedec, Adil Zouitine, Thomas Wolf, et al. LeRobot: State-of-the-art machine learning for real-world robotics in pytorch. <https://github.com/huggingface/lerobot>, 2024.
- [2] Jiahong Chen, Jing Wang, Long Chen, Chuwei Cai, and Jinghui Lu. NanoVLA: Routing decoupled vision-language understanding for nano-sized generalist robotic policies. *arXiv preprint arXiv:2510.25122*, 2025.
- [3] Kang Chen, Zhihao Liu, Tonghe Zhang, Zhen Guo, Si Xu, Hao Lin, et al. π RL: Online rl fine-tuning for flow-based vision-language-action models. *arXiv preprint arXiv:2510.25889*, 2025.
- [4] Cheng Chi, Siyuan Feng, Yilun Du, Zhenjia Xu, Eric Cousineau, Benjamin Burchfiel, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *Robotics: Science and Systems (RSS)*, 2023.
- [5] Jiajun Fan, Shuaike Shen, Chaoran Cheng, Yuxin Chen, Chumeng Liang, and Ge Liu. Online reward-weighted fine-tuning of flow matching with wasserstein regularization. *arXiv preprint arXiv:2502.06061*, 2025.
- [6] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, et al. OpenVLA: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.
- [7] Haozhan Li, Yuxin Zuo, Jiale Yu, Yuhao Zhang, Zhaohui Yang, Kaiyan Zhang, et al. SimpleVLA-RL: Scaling vla training via reinforcement learning. *arXiv preprint arXiv:2509.09674*, 2025.
- [8] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. In *International Conference on Learning Representations (ICLR)*, 2023.
- [9] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2023.

- [10] Jie Liu, Gongye Liu, Jiajun Liang, Yangguang Li, Jiaheng Liu, Xintao Wang, Pengfei Wan, Di Zhang, and Wanli Ouyang. Flow-GRPO: Training flow matching models via online rl. *arXiv preprint arXiv:2505.05470*, 2025.
- [11] Mingyang Lyu, Yinqian Sun, Erliang Lin, Huangrui Li, Ruolin Chen, Feifei Zhao, and Yi Zeng. Reinforcement fine-tuning of flow-matching policies for vision-language-action models. *arXiv preprint arXiv:2510.09976*, 2025.
- [12] Xue Bin Peng, Aviral Kumar, Grace Zhang, and Sergey Levine. Advantage-weighted regression: Simple and scalable off-policy reinforcement learning. *arXiv preprint arXiv:1910.00177*, 2019.
- [13] Jan Peters and Stefan Schaal. Reinforcement learning by reward-weighted regression for operational space control. In *International Conference on Machine Learning (ICML)*, 2007.
- [14] Allen Z. Ren, Justin Lidard, Lars L. Ankile, Anthony Simeonov, Pulkit Agrawal, Anirudha Majumdar, Benjamin Burchfiel, Hongkai Dai, and Max Simchowitz. Diffusion policy policy optimization. *arXiv preprint arXiv:2409.00588*, 2024.
- [15] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [16] Mustafa Shukor, Dana Aubakirova, Francesco Capuano, et al. SmolVLA: A vision-language-action model for affordable and efficient robotics. *arXiv preprint arXiv:2506.01844*, 2025.
- [17] Tonghe Zhang, Chao Yu, Sichang Su, and Yu Wang. ReinFlow: Fine-tuning flow matching policy with online reinforcement learning. *arXiv preprint arXiv:2505.22094*, 2025.